Fresh JMLR MLOSS reviewer report: kodama-cpp

Independent first-read assessment of the current submission package

Review date: 27 July 2026

Summary

This submission presents a substantial standalone implementation of KODAMA rather than a new learning objective. The manuscript clearly distinguishes continuity with the published KODAMA method from the new C++17 systems contribution: float32 state ownership, CPU/CUDA/Metal backends, label-aware SIMPLS plus latent-space LDA, package-owned neighbor search, supplied-graph entry points, and thin R/Python bindings. The algorithm is reproducible from the equations and pseudocode: proposal generation, one-CV-pass-per-cycle evaluation, acceptance score, cooling rule, independent M runs, landmark and splitting semantics, projection, and agreement-graph correction are all specified.

The empirical section is appropriately cautious. Internal CV accuracy is not presented as external truth, reference labels are used only after optimization, and positive and negative visualization results are retained. Kernel, optimizer, and end-to-end timings are separated. A legacy classifier-matched comparison and a separately scoped current-CRAN systems comparison prevent the predecessor discussion from relying on an obsolete release alone.

Major Comments

1. The reviewed software is still a release candidate rather than an immutable reviewed release. There is no public v0.1.0 tag or GitHub release, and no archived source snapshot. Freeze the exact reviewed commit, create the source archive required by MLOSS, publish its checksum, and cite that identity consistently. A DOI is useful but not itself mandatory.
2. Current-release end-to-end evidence is too narrow for the breadth of the CPU/CUDA/Metal claims. The matched full KODAMA M=100, Tcycle=100 result is presently MetRef only, while broader rows combine kernel tests, earlier runs, or different hardware. Rerun a predeclared multi-dataset matrix from the tagged release, preserve repeated wall-time and memory distributions, and report failures and unfavorable quality results unchanged.
3. CPU source coverage of 63.58% by line and 57.03% by branch is well below the MLOSS expectation of coverage close to 100%. Add tests for proposal and acceptance state transitions, degeneracy guards, graph transforms, PLS numerical-failure paths, and wrapper error contracts. CUDA is not exercised by public CI; archive hardware logs or add a maintained GPU runner.
4. The cover letter documents substantial use of the predecessor KODAMA R package, but this is not evidence of an active community around kodama-cpp. The new repository currently has no stars, forks, releases, public issues, or independently documented adopters. Obtain and cite verifiable beta-user, downstream-package, issue, or external-reproduction evidence rather than transferring predecessor adoption to the new implementation.
5. The tested Python binding is public only as an embedded source subtree, while the project architecture and submission materials describe independently versioned wrappers. Publish the standalone Python repository before submission, test installation from that public source in a clean environment, and state clearly which wrapper commit belongs to the reviewed core release.

Minor Comments

1. Explain timing boundaries, warm-up, synchronization, data-transfer inclusion, thread counts, and hardware beside the unusually large 425x Metal KNNCV result. The supplement contains much of this context, but the evidence table should point to one reproducible record.
2. Complete the author-only cover-letter fields: coauthor consent, funding, competing interests, and conflict-free editor/reviewer suggestions. Refresh all dated repository and download metrics immediately before submission.
3. Keep the current-CRAN comparison explicitly labeled as a short end-to-end systems check. KODAMA 3.3 automatically chooses its PLS route and is not classifier- or trajectory-parity with kodama-cpp PLS-LDA. The full M=100/Tcycle=100 legacy and sensitivity evidence should remain separate.

4. Document the native CUDA graph-builder limit k <= 256 prominently. Current CRAN couples a large landmark request to k=500 on MetRef, so an exactly matched CUDA row is correctly omitted rather than produced with a silent parameter change.
5. Retain the explicit distinction between internal CV accuracy, the search score, and external-label diagnostics. The current paper handles this well and should not be simplified into a claim that maximizing CV accuracy necessarily recovers reference classes.
6. Keep the four-page paper self-contained while treating the long technical document as a supplement. The present layout now satisfies the four-description-page limit, with references beginning on page 5.

Strengths

- The distinction between raw CV accuracy, proposal acceptance score, and external diagnostics is unusually clear and prevents circular quality claims.
- The single-run and ensemble pseudocode make the stochastic optimization independently implementable.
- Backend identity is strict and testable; unavailable accelerators do not masquerade as successful CPU execution.
- The evidence matrix links implementation claims to source locations, tests, and measured results.
- Cross-platform GitHub Actions, measured CPU coverage, a frozen API test, provenance audit, and hosted documentation substantially strengthen MLOSS readiness.
- The manuscript retains unfavorable datasets and states that KODAMA correction is classifier- and dataset-dependent.

Reviewer Recommendation

Recommendation: Major Revision. The software is suitable in scope and technically promising for JMLR MLOSS, and the four-page description is now clear and reproducible. However, the reviewed artifact is not yet frozen, coverage is substantially below the track's stated expectation, public CUDA evidence and current-release end-to-end validation remain incomplete, and active adoption of the new library has not yet been demonstrated.

I would encourage resubmission after these software-release and evidence issues are addressed. I do not request a new KODAMA objective or an algorithmic redesign: the principal work is to freeze and test the release, strengthen coverage and accelerator auditability, publish the Python wrapper, and establish independent usage evidence.